

Fit for the Engine? Auditing Image-to-3D Generated Meshes in a Shipped Game

A Pre-Registered Audit of 26 Generated Assets

Penggan Zhao
Independent Researcher
China
pgzhao101@gmail.com

Abstract

Image-to-3D generators let a solo developer populate a game with textured models from single drawings, and their output is judged on how it looks. A real-time engine judges other things. We audited the 26 generated assets shipped in a finished Unity game (24 static models from one Hunyuan 3D setting, a companion we simplified, and a rigged, animated boss) against a plan committed before any audit script ran. Of five hypotheses, two held and three did not. The raw meshes are clean: all 24 are watertight and edge-manifold. Every asset arrives scaled to about one unit, facing away from the viewer, with three 4096² maps whose memory is 40–50 times that of the geometry in BC7, and with half of each texture unused. The defects that cost time were conventions the file does not state, an animation that lifts the boss 0.46–0.68 m off the ground at game scale, and our own processing: simplifying the seam-split vertex buffer a glTF loader returns tore the companion into 2,433 pieces, while welding first leaves every asset crack-free at the same budgets. Three of the game’s six asset fixes repaired steps we had added. We release the scripts, measurements and labelled commit history, and give a short pre-import checklist ordered by what each check would have caught.

Keywords

image-to-3D generation, game assets, mesh simplification, real-time rendering, asset pipelines, content audit

1 Introduction

Image-to-3D generators now turn a single drawing into a textured mesh in a minute or two [25, 30]. For a solo developer this changes what is affordable: the 15 non-player characters and nine props of the game studied here came from one drawing each, where hand modelling would have taken weeks. Generators are evaluated on how their output looks, against the input image or by human preference. A real-time engine judges a mesh on other things: what size it is, which way it faces, where its origin is, whether its surface is closed and manifold, what it costs in memory, whether it survives simplification, and, once rigged, where its feet are when it moves.

This paper audits the generated meshes that shipped in a finished game, ChronoTraveler, against those engine-side properties. We ask what such an asset needs before it is fit for a game, and which of those needs could be found by counting before import rather than discovered in play. The audit plan was committed before any audit script was run. The corpus is small (24 assets from one pipeline, a simplified companion and a rigged boss) and comes from one

generator, so this is a case study; what transfers is the checklist and how it was tested.

The findings run against the expectations we wrote down. The raw meshes are clean: all 24 are watertight and manifold. The defects that cost the most were either invisible to a file inspection (the generator’s characters face away from the viewer; the rigged boss’s idle animation lifts it 0.46–0.68 m off the ground at game scale) or introduced by our own processing. Our simplification of the companion tore it into 2,433 pieces, because the simplifier was given the seam-split vertex buffer that a glTF loader returns; welding the same input first removes every crack. Memory is set by the textures: three 4096² maps cost 40–50 times the geometry in BC7, so the “50k faces” setting users choose controls the cheaper part.

Contributions: (1) a pre-registered audit of generated meshes in a shipped game, with every script, metric and commit label released; (2) evidence that, for this pipeline, engine readiness is decided by conventions, texture budget, simplification input and animation rather than by mesh topology; (3) a short pre-import checklist ordered by what each check would have caught here (Section 6). It extends our earlier audits of LLM-generated game content [28, 29] from text to geometry.

2 Related Work

Image-to-3D generators. Feed-forward and diffusion-based systems now produce textured meshes from one image in seconds to minutes: TripoSR [21], InstantMesh [26], SF3D [3], TRELLIS [25] and the Hunyuan3D series [18, 30], whose commercial service generated our assets [19]. Several describe their output as production- or game-ready; Hunyuan3D Studio adds retopology, UV unwrapping and rigging for that purpose [13], and our boss came from it. These claims are made and checked by the developers of each system.

Evaluating generated 3D. Benchmarks such as T3Bench [10], GPTEval3D [24] and 3DGen-Bench [27] score rendered views for plausibility, prompt alignment and human preference. A second line, from MeshGPT [17] to MeshAnything [4, 5], argues that densely extracted meshes are wasteful for storage and rendering and changes the generator’s output representation toward artist-like topology. We found no independent audit of a generator’s output after it enters a real-time engine, covering units, axes, pivots, UV efficiency, memory and animation. We searched arXiv and the web rather than every venue, so we claim only that such work is not prominent.

Table 1: Audited assets. “Raw” = generator output used as downloaded.

Group	<i>n</i>	Source and use
NPCs	15	image-to-3D, 50k setting; raw GLB, static
Props	9	same pipeline; raw GLB, static
Companion (raw)	1	image-to-3D, high setting; 497,850 triangles
Companion (shipped)	1	our UV-preserving simplification; 82,829 triangles
Boss	1	studio pipeline: retopology, rig, 4 animations (4 FBX)

Mesh processing. Our measurements use standard tools: quadric error simplification [8] and its extensions to appearance attributes [9, 11], texture-aware simplification [16], Hausdorff error as computed by Metro [7], and MeshLab [6] through PyMeshLab [15]. Attene et al. survey mesh defects and repair from an application perspective [2]; our hygiene checks cover a subset of the defect classes they list. Texture formats and their costs follow standard references [1, 14], and the asset format is glTF 2.0 [20].

Grounding animated characters. Keeping feet on the ground during playback is usually a per-frame problem, handled by foot-contact cleanup or inverse kinematics [12]. Engines expose bounds for skinned meshes that are not recomputed from the current pose [22], and a pose snapshot API whose scale handling depends on the hierarchy [23]; both figure in our results.

3 The Assets

ChronoTraveler is a turn-based educational RPG built in Unity 6 (URP) by one developer working with an AI coding assistant over two months in 2026. Its five historical eras each have three non-player characters (NPCs); the Silk Road era also has nine props (two camels, a horse, a cart, a stupa, a milestone, silk bolts and spice sacks). All 24 were generated from a single front-view image each with Tencent’s Hunyuan 3D web service (image-to-3D, version 3.1, “50k faces” setting) and downloaded as GLB files. Two further assets came from the same service by other routes: the companion fox, generated at a higher face setting (497,850 triangles) and then simplified by us to 82,829 triangles, and the final boss, produced by the service’s studio pipeline, which adds retopology, UV unwrapping, texturing, a 28-bone rig and four animations, delivered as four FBX files sharing one 9,038-triangle mesh. Table 1 summarises the corpus; `data/manifest.json` lists every file with its SHA-256.

The 24 GLBs in the shipped game are byte-identical to the downloads. Nothing was edited before import. Every correction was applied inside the engine: an editor tool rescales each NPC to 1.7 m, turns it 180° and converts its material; a second tool does the same for the boss at 10 m; a run-time component (GroundSnap) moves the animated boss down onto its platform.

4 Method

The audit plan—metrics, hypotheses and decision rules—was committed and pushed to the study repository (commit 2b0bc54) before any audit script was written. One disclosure: while scoping the study the same day, a read-only survey had already parsed the

GLB headers and reported triangle counts, texture sizes, the double-sided flag, the root rotation and unit-scale heights. The plan says so and treats the hypothesis that restates them (H1) as a descriptive confirmation, not a test. The hygiene, simplification, history and grounding results had not been looked at. Deviations are listed in Section 8.

B1 Conventions. A small GLB reader (no glTF library, so the numbers come from the same bytes the importer reads) applies each file’s node chain and records the bounding box, the largest extent, the pivot as a fraction of the box on each axis, node transforms, material flags and extensions, and each embedded image’s format and pixel size. **H1:** every raw asset is scale-normalised (largest extent 0.9–1.2 units), carries a non-identity root rotation and is double-sided.

B2 Geometry hygiene. Topology is measured after welding vertices with identical positions, since a UV seam is not a hole: connected components, boundary and non-manifold edges, non-manifold vertices, degenerate and duplicate faces, self-intersecting faces (MeshLab [6] via PyMeshLab [15]), and genus. UV charts are counted on the unwelded mesh, where seams separate them; UV coverage is the union of the UV triangles rasterised at 2048². **H2:** a majority of the 24 raw assets fail at least one of watertight, edge-manifold, or single component.

B3 Cost. Geometry bytes as stored (vertex attributes plus indices) and texture bytes for three GPU formats, each with a full mip chain (×4/3): uncompressed RGBA8 (4 B/pixel), BC7 (1 B/pixel) and BC1 (0.5 B/pixel). **H3:** texture memory exceeds geometry memory at least tenfold for every raw asset under every format.

B4 Simplification. Quadric edge collapse [8], in its texture-aware form [9, 11], as implemented in MeshLab, with quality threshold 0.3 and boundary preservation off, at 50, 25, 12.5 and 6.25 % of the original face count, up to three passes. The plan named a with-UV and a no-UV condition. Because B2 showed that the shipped companion had been torn (Section 5.2), the with-UV condition was run on two input representations of the same file: *S*, the seam-split vertex buffer that MeshLab’s glTF loader returns (what our companion pipeline used), and *W*, the same mesh with exact duplicates welded and UVs kept per corner. *N* is welded with no UVs. For each run we record faces reached, open edges after welding, components, and sampled Hausdorff distance [7] in both directions as a percentage of the bounding-box diagonal. **H4:** with UVs preserved, a majority of assets cannot reach 6.25 % within three passes, while the no-UV control can.

B5 History. All non-merge commits in the game repository up to the shipped version (484) were searched with a pre-registered case-insensitive pattern of 25 terms (scale, pivot, ground, clip, cull, bounds, glb, decimat, ...). Each of the 156 matches was labelled from its message and diff as ASSET (root cause is a property of a generated asset), ENGINE (an engine or tool behaviour any asset would hit), LEVEL (level geometry, placement, purchased packs, procedural content) or OTHER. Two AI labellers drafted the labels, each taking half; the AI assistant reviewed every ASSET, ENGINE and low-confidence label, added an *origin* field (did the generator ship the defect, or did our own processing introduce it?) and recorded

each override with its reason; the author signed off. For ASSET fixes we note which B1–B3 check, if any, would have flagged the property before import. **H5**: most ASSET fixes concern properties that counting before import (B1/B2) would reveal.

E1 Grounding (exploratory). The boss was mis-grounded five times in 26 minutes of development. In Blender 5.1 we evaluate the skinned mesh at every frame of each animation and take the lowest vertex as ground truth, then score the estimators the game tried: bind-pose bounds, a single frame, and the shipped minimum over ten samples 0.16 s apart. Errors are reported at game scale (10 m).

5 Results

5.1 Conventions: every asset needs the same three corrections

H1 holds with one marginal exception. All 24 raw assets carry the same root rotation, $+90^\circ$ about X (the mesh is stored Z-up and rotated into glTF’s Y-up), all are double-sided, and all use the same material layout: one primitive, one material with KHR_materials_specular, and three 4096² PNG maps (base colour, normal, metallic-roughness). Every file names Blender’s glTF exporter as its generator. Scale is normalised regardless of subject: the largest extent is 0.895–1.151 units (median 1.082), so a camel, a cart, a stupa and a soldier all arrive about one unit long. One asset, the milestone, falls 0.005 below the pre-registered 0.9 bound. The pivot sits on the base of the bounding box in every case (vertical fraction 0.000) and near, but not at, its horizontal centre (0.30–0.69 of the depth).

None of this is a defect in itself. Feet at the origin is the convention a game wants, and a $+90^\circ$ node is correct glTF. But two of the three corrections the game needed for every asset cannot be derived from the file: real-world size must come from outside, and the facing direction ($-Z$, so the character shows its back to a player standing in front of it) is not encoded anywhere. The only property a counting check could not see is the one that caused the first fix.

5.2 Hygiene: the generator’s meshes are clean; ours was not

H2 is not supported. All 24 raw assets are watertight and edge-manifold, with no degenerate or duplicate faces; 19 are a single connected component and five consist of two to four (Wilson 95% CI for failing any of the three checks: 9–41%). What every asset does have is self-intersections (11–524 faces, median 64) and a high genus (median 60, up to 361), consistent with small tunnels in a surface extracted from an implicit field. None of these showed up in play.

The UV layouts are where the meshes depart from hand-made assets. The 24 raw assets have 703–3,088 UV charts each (median 1,568), and the union of their UV triangles covers 48–68 % of the texture square (median 53 %): about half of every 4096² map stores nothing. Charts do not overlap. Seams split vertices 1.31–1.81 times over (median 1.53), so the vertex buffer the GPU receives is half again as large as the surface needs.

The one mesh that fails every check is the one we processed. The companion’s raw 497,850-triangle file is watertight, manifold and a single component, like the others. The shipped 82,829-triangle

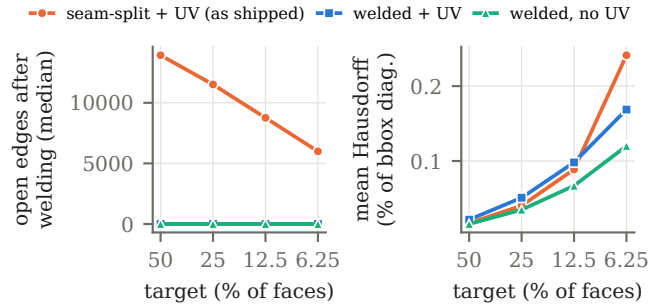


Figure 1: Simplifying the 24 raw assets (medians). Left: open edges after welding coincident vertices; the inputs have none. Seam-split input (as our companion pipeline used) cracks at every target; welded input, with or without UVs, never does (the two lines coincide at zero). Right: mean Hausdorff error.

version has 2,433 components, 44,316 open edges and 1,442 non-manifold vertices; welding at tolerances up to 10^{-4} of the diagonal closes none of them. Section 5.4 shows why.

5.3 Cost: the texture is the asset

H3 holds for all 24 assets. Their geometry occupies 1.28–1.60 MiB as stored; their textures occupy 256 MiB uncompressed with mips, 64 MiB in BC7 and 32 MiB in BC1. Texture memory is 20–25 times the geometry even in the smallest format, and 40–50 times in BC7, the usual choice for colour maps on desktop GPUs [1]. The “50k faces” setting a user chooses in the generator controls the cheaper part. The hypothesis fails for the one asset generated at a higher face setting: at 497,850 triangles the companion’s ratio falls to 4.3 (BC7) and 2.1 (BC1). On disk the 24 files total 852 MiB.

A consistency check from the shipped build (added after the plan). The game’s compliance benchmark (July 2026, same machine) logged Unity’s total allocated memory per scene. The four eras with three generated NPCs each sit at 403–424 MiB; the Silk Road, which loads the same three NPCs plus the nine generated props, sits at 963 MiB, about 550 MiB higher, or 61 MiB per extra prop. That is close to the BC7 estimate of 64 MiB and far from the 256 MiB an uncompressed import would need. The Silk Road scene also has other procedural content and the benchmark did not isolate these assets, so this is consistency, not measurement. The game’s own benchmark summary had attributed the excess to procedural market objects.

5.4 Simplification: the input, not the UVs, decides the damage

H4 is not supported. With UVs preserved on a welded mesh, 23 of 24 assets reached 6.25 % of their faces (about 3,100) in a single pass, with no cracks (Table 2). The exception is the asset with the most UV charts (3,088), which stalled at 3,896 faces after three passes. Charts constrain where the simplifier may collapse, but at these budgets they rarely stop it. What decided the outcome was the input representation. MeshLab’s glTF loader returns the file’s vertex buffer, in which every UV seam has already split the surface into separate

Table 2: Simplification at 6.25 % of the original faces (24 assets): targets reached within three passes, assets left with open edges, and Hausdorff error as % of the bounding-box diagonal (median [max]).

Input	Reached	Cracked	Mean err.	Max err.
S: seam-split + UV	18	24	0.24 [0.32]	3.6 [8.3]
W: welded + UV	23	0	0.17 [1.40]	1.4 [9.7]
N: welded, no UV	24	0	0.12 [0.20]	1.6 [5.5]

vertices. Given that, texture-aware simplification treats each seam as an open boundary and moves its two sides independently: every asset cracked at every target, with a median of 13,924 open edges and 954 components already at 50 %, and six assets could not reach 6.25 % at all (Figure 1). Nothing in the filter reports this; the output is a valid mesh with the requested face count, and it renders. Welding exact duplicates first, one call, removes every crack at the same face budgets, with comparable mean error (medians within 0.08 % of the diagonal at every target).

This is what happened to the companion. Our pipeline loaded the 497,850-triangle GLB directly and simplified it in three passes to 82,829 triangles; the result is the torn mesh of Section 5.2. The design note for that step recorded that simplification “converged” near 106k triangles because the generator’s UV islands were a hard floor. The data say otherwise: $82,829$ is $497,850 \times 0.55^3$, the loop’s per-round factor applied three times, so the script stopped because it ran out of rounds, not because it hit a floor. On the welded input the same filter takes the companion to 31,116 triangles (6.25 %) in one pass with no open edges and a mean error of 0.014 % of the diagonal; the seam-split input reaches the same count with 34,624 open edges. It shipped after a play check in which no one reported a crack; the defect was found by this audit.

5.5 History: three of six asset fixes repaired our own processing

Of 484 non-merge commits, 156 matched the pattern. After review they split into 61 LEVEL, 76 OTHER, 6 ENGINE and 13 ASSET. Generated assets were a small part of the game’s spatial trouble: level geometry, purchased packs and procedural environment account for nearly five times as many commits.

Seven of the 13 ASSET commits are not fixes but tooling written ahead of need: the NPC import tool (scale to 1.7 m, turn 180°, convert material), its application to each era’s models, the boss bake tool and the companion’s simplification and pivot setup. For the 24 static assets, scale and pivot never became bugs: the developer noted them on the first imported file and built the correction into the import step. The six fixes are a different list. Two repaired facing (the $-Z$ front, then a second 180° turn that cancelled the first); two repaired the companion lying on its back after our GLB-to-OBJ conversion dropped the root rotation node; one was the boss hovering above its platform; one was a scale calibration for the same problem. By origin, three were shipped by the generator and three were introduced by our own processing (the double turn and the lost rotation node, twice). Only three of the six concern a property a metadata count would reveal, so H5 is not supported: facing (two

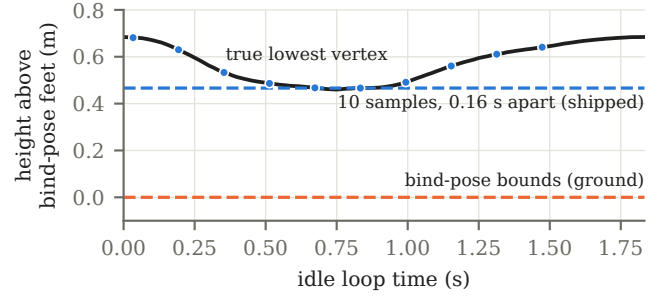


Figure 2: Boss idle loop at game scale (10 m). Placing the bind-pose bounds on the ground leaves the animated feet 0.46–0.68 m in the air for the whole loop. The shipped estimator (minimum of ten samples) lands within 6 mm of the loop’s true minimum; the remaining 0.22 m is the idle bob itself.

fixes) is invisible to counting, and grounding (the longest chain of fixes) needed the animation to be evaluated.

5.6 Grounding: the bind pose lies, and any constant has a floor

The boss’s studio mesh is skinned so that its bind pose stands with its feet at the origin, but its idle animation lifts the whole body (Figure 2). Snapping the bind-pose bounds to the ground, the engine’s default and the game’s first attempt, leaves the animated feet 0.46–0.68 m above the platform on every frame. A single sampled frame can land anywhere in the 0.22 m bob. The shipped estimator, the minimum of ten samples 0.16 s apart, reaches the loop’s minimum to within 6 mm in Blender. What remains is not estimator error: a model moved by one constant offset cannot follow a body that moves up and down, so 0.22 m of hover at the top of each bob is the floor for any constant, and only per-frame contact handling (foot IK [12]) goes below it. In the engine the same estimator needed a further empirical 0.30 m because Unity’s BakeMesh returned two incompatible scalings for this rig (0.68 and 2.29 against a hover of about 0.9 bracketed in play); we report those figures from the commit record and did not reproduce them.

6 Discussion

The mesh was the easy part. We expected generated meshes to fail as meshes: holes, non-manifold edges, stray fragments. They did not. The problems that cost development time were conventions that a file does not state (size, facing), a texture budget set by the generator rather than the user, a processing step whose input representation we did not check, and an animation whose pose differed from its bind pose. Three of the six fixes repaired our own processing, not the generator’s output, and the one simplification step we added both tore the mesh and rested on a wrong diagnosis: its “UV floor” was a loop that ran out of rounds. For a developer adopting such a pipeline, the lesson is less “inspect the generator” than “inspect every step you add after it, and the explanation you wrote down for it”.

A pre-import checklist. Ordered by what each check would have caught in this game, at the cost of a script that runs in seconds per asset:

- (1) *Size and facing.* Read the largest extent and treat it as unitless; supply real size from the design. Facing is not in the file: render one thumbnail from +Z, or adopt the generator’s convention once and test it on the first asset.
- (2) *Texture budget.* Count texels, not triangles. Three 4096² maps cost 64 MiB in BC7 per asset; most of the half-empty atlas can be dropped by downscaling or re-baking.
- (3) *Simplification input.* Weld before simplifying, and check open edges after. A seam-split buffer looks like a mesh and simplifies without error.
- (4) *Animated contact.* For rigged assets, evaluate each loop offline and compare its lowest vertex with the bind pose; budget the bob amplitude, or use foot contact handling.
- (5) *Topology.* Watertightness and manifoldness were never the problem here, but they are cheap to count and our own tool was the one that broke them.

Counting versus looking. Our earlier audit of an LLM-generated quiz bank found that the defects that matter were invisible one item at a time and obvious when counted across the set [28]. Generated meshes partly reverse this. Counting confirmed that every asset shares the same conventions, so one correction serves all of them, and counting found the torn companion and the texture budget. But the costliest defects, facing and grounding, were found by looking at the game, and could only have been found earlier by looking: rendering a view, or evaluating an animation. A useful pre-import check for generated geometry is a small render and a short simulation, not only a statistic.

7 Limitations

Scope. One generator, one version, one game: 24 assets from one setting plus two from other routes. Hunyuan 3D’s web service is closed and changes over time; another generator, or this one a few months later, may differ on every property we measured. The 50k-face setting and 4096² textures were the defaults we used, not a sweep.

No frame-time measurements. Memory figures are estimates from texel counts and formats; the one in-game cross-check is a scene-level total that includes other content. We did not profile draw calls, frame time or GPU memory per asset, and the game’s benchmarks never isolated these assets.

Labelling. The 156 history labels were drafted by two AI labellers and reviewed by the AI assistant, with the author signing off; there was no second human rater. Commit messages describe what the developer believed at the time, so root causes inherit their mistakes (for example, a fix attributed to idle “bob” that the next commit disproved). Keyword search misses commits that never used the words.

Grounding. Ground truth comes from Blender’s evaluation of the FBX, not from the engine. The Unity-side scaling ambiguity is reported from the commit record and was not reproduced. The boss is one rig with four animations.

Quality not scored. We did not measure visual fidelity to the input image, texture quality, or how players perceived any of the assets.

Authorship. The author developed the game with an AI coding assistant, and the same assistant helped design and run this audit and draft the paper. Plan, scripts, outputs and labels are public so that the numbers can be checked without trusting either.

8 Deviations from the Plan

- B4 was planned with one with-UV condition. After B2 found the shipped companion torn, the with-UV condition was run on two input representations (seam-split *S*, welded *W*); both are reported.
- B5 labels were drafted by two AI labellers splitting the commits in half, rather than one; the reviewer added an *origin* field not in the plan and changed one precheck value (the boss hover, from “none” to per-frame animation, because E1 shows an offline check finds it). Both are recorded with reasons in `results/history_labels.json`.
- The scene-memory consistency check (Section 5.3) was not planned.
- The GLB reader and the hygiene script were run before the B4 script was written; no hypothesis or threshold was changed after any result was seen.

Acknowledgments

The game, its import tools and this audit were built with the help of an AI coding assistant (Anthropic’s Claude in Claude Code), which also drafted the commit labels, the analysis scripts and parts of this text; the author reviewed every label, number and sentence. Assets were generated with Tencent’s Hunyuan 3D service. No external funding.

References

- [1] Tomas Akenine-Möller, Eric Haines, Naty Hoffman, Angelo Pesce, Michal Iwanicki, and Sébastien Hillaire. 2018. *Real-Time Rendering* (4th ed.). A K Peters/CRC Press, Boca Raton, FL. doi:10.1201/9781315365459
- [2] Marco Attene, Marcel Campen, and Leif Kobbelt. 2013. Polygon Mesh Repairing: An Application Perspective. *Comput. Surveys* 45, 2 (2013), 1–33. doi:10.1145/2431211.2431214
- [3] Mark Boss, Zixuan Huang, Aaryaman Vasishta, and Varun Jampani. 2025. SF3D: Stable Fast 3D Mesh Reconstruction with UV-unwrapping and Illumination Disentanglement. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 16240–16250. arXiv:2408.00653. https://openaccess.thecvf.com/content/CVPR2025/html/Boss_SF3D_Stable_Fast_3D_Mesh_Reconstruction_with_UV-unwrapping_and_Illumination_CVPR_2025_paper.html
- [4] Yiwen Chen, Tong He, Di Huang, Weicai Ye, Sijin Chen, Jiaxiang Tang, Xin Chen, Zhongang Cai, Lei Yang, Gang Yu, Guosheng Lin, and Chi Zhang. 2025. MeshAnything: Artist-Created Mesh Generation with Autoregressive Transformers. In *International Conference on Learning Representations (ICLR)*. arXiv:2406.10163. https://proceedings.iclr.cc/paper_files/paper/2025/hash/7fc69f53a6ca2664ef23a9597903ca6f-Abstract-Conference.html
- [5] Yiwen Chen, Yikai Wang, Yihao Luo, Zhengyi Wang, Zilong Chen, Jun Zhu, Chi Zhang, and Guosheng Lin. 2024. MeshAnything V2: Artist-Created Mesh Generation With Adjacent Mesh Tokenization. arXiv:2408.02555 [cs.CV] doi:10.48550/arXiv.2408.02555
- [6] Paolo Cignoni, Marco Callieri, Massimiliano Corsini, Matteo Dellepiane, Fabio Ganovelli, and Guido Ranzuglia. 2008. MeshLab: an Open-Source Mesh Processing Tool. In *Eurographics Italian Chapter Conference*. The Eurographics Association, 129–136. doi:10.2312/LocalChapterEvents/ItalChap/ItalianChapConf2008/129-136

- [7] Paolo Cignoni, Claudio Rocchini, and Roberto Scopigno. 1998. Metro: Measuring Error on Simplified Surfaces. *Computer Graphics Forum* 17, 2 (1998), 167–174. doi:10.1111/1467-8659.00236
- [8] Michael Garland and Paul S. Heckbert. 1997. Surface Simplification Using Quadric Error Metrics. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '97)*. ACM Press/Addison-Wesley, 209–216. doi:10.1145/258734.258849
- [9] Michael Garland and Paul S. Heckbert. 1998. Simplifying Surfaces with Color and Texture Using Quadric Error Metrics. In *Proceedings Visualization '98*. IEEE, 263–269. doi:10.1109/VISUAL.1998.745312
- [10] Yuze He, Yushi Bai, Matthieu Lin, Wang Zhao, Yubin Hu, Jenny Sheng, Ran Yi, Juanzi Li, and Yong-Jin Liu. 2023. T³Bench: Benchmarking Current Progress in Text-to-3D Generation. arXiv:2310.02977 [cs.CV] doi:10.48550/arXiv.2310.02977
- [11] Hugues Hoppe. 1999. New Quadric Metric for Simplifying Meshes with Appearance Attributes. In *Proceedings Visualization '99*. IEEE, 59–66. doi:10.1109/VISUAL.1999.809869
- [12] Lucas Kovar, John Schreiner, and Michael Gleicher. 2002. Footskate Cleanup for Motion Capture Editing. In *Proceedings of the 2002 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '02)*. ACM, 97–104. doi:10.1145/545261.545277
- [13] Biwen Lei, Yang Li, Xinhai Liu, Shuhui Yang, Lixin Xu, Jingwei Huang, Ruining Tang, Haoan Weng, Jian Liu, Jing Xu, et al. 2025. Hunyuan3D Studio: End-to-End AI Pipeline for Game-Ready 3D Asset Generation. arXiv:2509.12815 [cs.CV] doi:10.48550/arXiv.2509.12815
- [14] Microsoft. 2026. BC7 Format (Direct3D 11). Microsoft Learn, Win32 apps documentation. Accessed 2026-10-05. <https://learn.microsoft.com/en-us/windows/win32/direct3d11/bc7-format>
- [15] Alessandro Muntoni and Paolo Cignoni. 2021. *PyMeshLab*. Concept DOI; version 2025.7.post1 used in this study. doi:10.5281/zenodo.4438750
- [16] Pedro V. Sander, John Snyder, Steven J. Gortler, and Hugues Hoppe. 2001. Texture Mapping Progressive Meshes. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '01)*. ACM, 409–416. doi:10.1145/383259.383307
- [17] Yawar Siddiqui, Antonio Alliegro, Alexey Artemov, Tatiana Tommasi, Daniele Sirigatti, Vladislav Rosov, Angela Dai, and Matthias Nießner. 2024. MeshGPT: Generating Triangle Meshes with Decoder-Only Transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 19615–19625. arXiv:2311.15475. https://openaccess.thecvf.com/content/CVPR2024/html/Siddiqui_MeshGPT_Generating_Triangle_Meshes_with_Decoder-Only_Transformers_CVPR_2024_paper.html
- [18] Team Hunyuan3D, Shuhui Yang, Mingxin Yang, Yifei Feng, Xin Huang, Sheng Zhang, Zebin He, Di Luo, Haolin Liu, Yunfei Zhao, et al. 2025. Hunyuan3D 2.1: From Images to High-Fidelity 3D Assets with Production-Ready PBR Material. arXiv:2506.15442 [cs.CV] doi:10.48550/arXiv.2506.15442
- [19] Tencent Cloud. 2026. Submit Hunyuan 3D Professional Job (SubmitHunyuanTo3DProJob) – Tencent Hunyuan 3D API Documentation. Official API documentation (in Chinese), last updated 2026-05-11. Accessed 2026-10-05. <https://cloud.tencent.com/document/product/1804/123447>
- [20] The Khronos 3D Formats Working Group. 2021. *glTF 2.0 Specification*. Specification Version 2.0.1. The Khronos Group Inc. <https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>
- [21] Dmitry Tochilkin, David Pankratz, Zexiang Liu, Zixuan Huang, Adam Letts, Yangguang Li, Ding Liang, Christian Laforte, Varun Jampani, and Yan-Pei Cao. 2024. TripoSR: Fast 3D Object Reconstruction from a Single Image. arXiv:2403.02151 [cs.CV] doi:10.48550/arXiv.2403.02151
- [22] Unity Technologies. 2026. *Renderer.localBounds*. Unity Scripting API. Accessed 2026-10-05. <https://docs.unity3d.com/ScriptReference/Renderer-localBounds.html>
- [23] Unity Technologies. 2026. *SkinnedMeshRenderer.BakeMesh*. Unity Scripting API. Accessed 2026-10-05. <https://docs.unity3d.com/ScriptReference/SkinnedMeshRenderer.BakeMesh.html>
- [24] Tong Wu, Guandao Yang, Zhibing Li, Kai Zhang, Ziwei Liu, Leonidas Guibas, Dahua Lin, and Gordon Wetzstein. 2024. GPT-4V(ision) is a Human-Aligned Evaluator for Text-to-3D Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 22227–22238. arXiv:2401.04092. https://openaccess.thecvf.com/content/CVPR2024/html/Wu_GPT-4Vision_is_a_Human-Aligned_Evaluator_for_Text-to-3D_Generation_CVPR_2024_paper.html
- [25] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. 2025. Structured 3D Latents for Scalable and Versatile 3D Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 21469–21480. arXiv:2412.01506. https://openaccess.thecvf.com/content/CVPR2025/html/Xiang_Structured_3D_Latents_for_Scalable_and_Versatile_3D_Generation_CVPR_2025_paper.html
- [26] Jiale Xu, Weihao Cheng, Yiming Gao, Xintao Wang, Shenghua Gao, and Ying Shan. 2024. InstantMesh: Efficient 3D Mesh Generation from a Single Image with Sparse-view Large Reconstruction Models. arXiv:2404.07191 [cs.CV] doi:10.48550/arXiv.2404.07191
- [27] Yuhang Zhang, Mengchen Zhang, Tong Wu, Tengfei Wang, Gordon Wetzstein, Dahua Lin, and Ziwei Liu. 2025. 3DGen-Bench: Comprehensive Benchmark Suite for 3D Generative Models. arXiv:2503.21745 [cs.CV] doi:10.48550/arXiv.2503.21745
- [28] Penggan Zhao. 2026. Counting What Inspection Misses: Auditing an LLM-Generated Quiz Bank for an Educational Game. Preprint. doi:10.5281/zenodo.23150165
- [29] Penggan Zhao. 2026. Feature or Leak? Guarding an LLM-Driven NPC Against Answer Leakage in an Educational Game. Preprint. doi:10.5281/zenodo.23150869
- [30] Zibo Zhao, Zeqiang Lai, Qingxiang Lin, Yunfei Zhao, Haolin Liu, Shuhui Yang, Yifei Feng, Mingxin Yang, Sheng Zhang, Xianghui Yang, et al. 2025. Hunyuan3D 2.0: Scaling Diffusion Models for High Resolution Textured 3D Assets Generation. arXiv:2501.12202 [cs.CV] doi:10.48550/arXiv.2501.12202